OPTIMIZING PYTHON

PROFILING AND OPTIMIZING

Optimize what needs optimizing

When should we think about optimizing our code

After it works, of course!

You should approach optimization like this:

1. Get it right (make sure your code works correctly)
2. Test that it’s right (validate and verify)
3. Profile if slow (fire up cProfile to identify bottlenecks)
4. Optimize (try to widen a bottleneck)
5. Repeat from 2 until you’re happy (or run out of time)

Step 1: eliminate warnings and errors

This sounds moot, but it's important

Your code won't compile and replace any existing pyc files in the interpreter cache if it doesn't compile

Removing warnings makes your code easier to read

PyCharm applies inspections against our code constantly and continuously to:

- detect and correct anomalous code in your project
- locate dead code
- find probable bugs, etc.

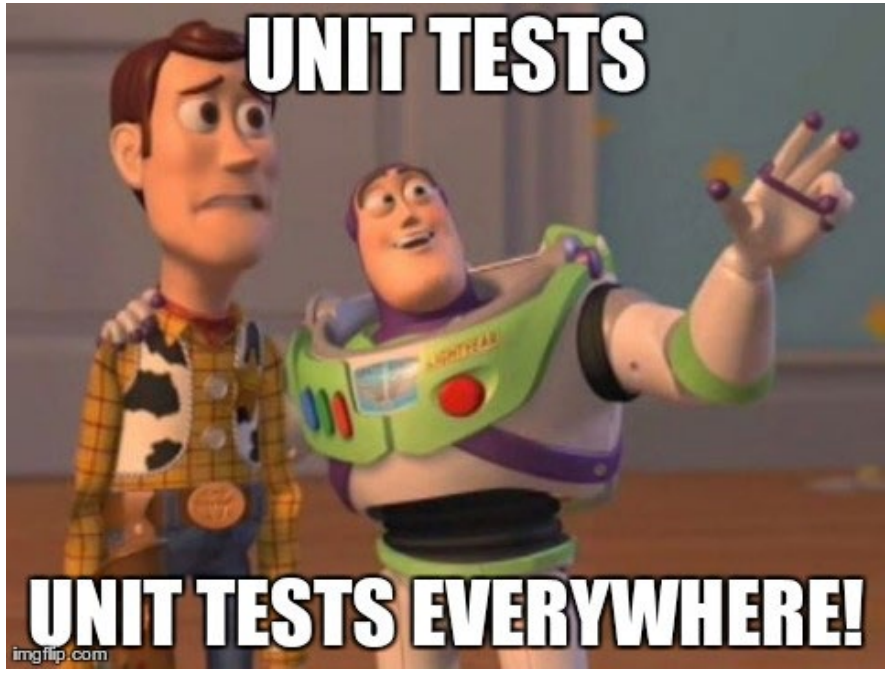
Step 2: validate and verify

Validation:

1. “Did we build the correct app?”
2. Demonstrate that the program meets its requirements

Verification:

1. “Did we build the app correctly?”
2. Discover situations in which the behavior of the software is incorrect or undesirable
3. Unit tests. Unit tests galore.

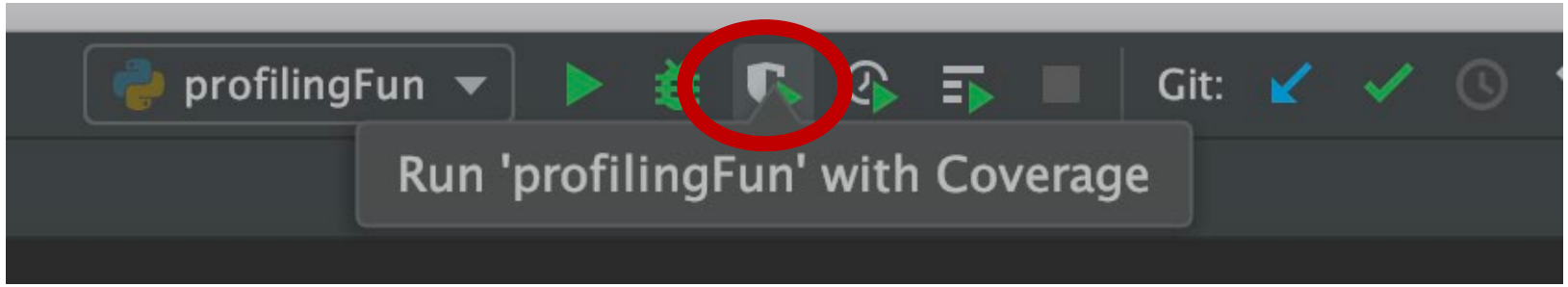


Step 3: remove unnecessary code

Run (profile) your program with coverage

This will reveal which functions are not being called

If there is code that is not being executed, consider removing it!



* https://www.jetbrains.com/help/pycharm/running-test-with-coverage.html

And now the fun stuff

It's time to apply a simple optimization algorithm:

1. **Measure** to identify slow code
   - Where is the hottest loop?
   - Where is the biggest, clumsiest object?
   - Which operation is most expensive?
2. **Alter (optimize)** the slow code to make it faster
3. **Verify** that the fix actually worked (and didn't slow anything else down)
4. **Repeat** until you run out of time!

Time complexity, aka use dicts and sets

Be familiar with the time complexity of Python's main data types

Check out: https://wiki.python.org/moin/TimeComplexity

In Python, dicts and sets use hash tables

- Hash tables have O(1) (CONSTANT!) lookup performance
- Hash tables can grow and shrink and this never takes longer!
- Long live the hash table!

Time complexity, aka use dicts and sets

Membership testing with sets and dictionaries is much faster, O(1), than searching sequences, O(n)

When testing "a in b", b should be a set or dictionary instead of a list or tuple.

Lists perform well as either fixed length arrays or variable length stacks

For queue applications using pop(0) or insert(0,v), collections.deque() offers superior O(1) performance

It avoids the O(n) step of rebuilding a full list for each insertion or deletion.

Reduce your memory footprint

Use generators when you can!

- We love lazy (on demand) generation of values
- This translates to lower memory usage.

Do you need an extra copy? Should you use the sort method or the sorted function?

Try them both out and see the impact when profiling!

__slots__* allow us to explicitly declare data members (like properties)

- denies the creation of __dict__ and __weakref__ (unless explicitly declared in __slots__ or available in a parent)
- the space saved over using __dict__ can be significant
- attribute lookup speed can be significantly improved as well.

* https://docs.python.org/3.7/reference/datamodel.html#slots

Use builtin functions and libraries

Built-in functions like sum( ), max( ), any( ), map( ), etc., are implemented in C

They are very efficient and well tested

Use them!

```
newlist = []
for word in oldlist:
    newlist.append(word.upper())
```

```
newlist = map(str.upper, oldlist)
```

* Check out the collections module for Pythonic and ***EFFICIENT*** data structures

Move calculations outside the loop

Suppose you have a big collection

You need to do some regex matching, i.e., match a date

Compile the regex once, or calculate the value, etc., outside of the loop, and use that 'cached' version in the loop.

You can also assign a function (calculation) to a local variable

Python accesses local variables much more efficiently than global variables

This is like ***memoization*** (an optimization technique that stores the results of expensive function calls and returns the cached result when the same inputs occur again)

Keep your codebase small

This is a bit of a pipe dream

Be conscious what you put in the module vs in the main method

When you import a module, there is the potential for a lot of code to run

This can slow down your program

Reduce the number of if/elif/and/or, etc.

Python's "Ask forgiveness" code style runs faster because it eliminates the need for the if check!

String concatenation: don’t use +

String concatenation is best done with ''.join(seq). If you don’t know how this works, now is a good time to look it up!

The join function is a O(n) process

In contrast, using the '+' or '+=' operators can result in an O(n**2) process

New strings may be built for each intermediate step

This generates too much overhead!

What else?

Avoid dots. Don’t chain function calls.

In functions, **local variables are accessed more quickly** than global variables, builtins, and attribute lookups

- It is worth localizing instance variable access in inner-loops
- For example, localize access to an instance variable with **local_pointer = self.my_thing**. That saves the shuffling loop from having to repeatedly lookup self.my_thing
- Outside of loops, the gain is minimal and rarely worth it.

Factor constant expressions out of loops

Inside loops, write "x = 3" instead of "x = 1 + 2"

What else?

Multiple assignment is slower than individual assignment

For example "x, y = a, b" is slower than "x = a; y = b”

However, multiple assignment is faster for variable swaps

For example, "x, y = y, x" is faster than "t = x; x = y; y = t"

Chained comparisons are faster than using the "and" operator, i.e., write "x < y < z" instead of "x < y and y < z”

What else?

Python import statements can be executed just about anywhere

It can be useful to place them inside functions to restrict their visibility and/or reduce initial start-up time (***lazy*** importing!). Although this is very rare and not the best practice. **This should be done only if necessary.**

Python's interpreter is optimized to not import the same module multiple times

But repeatedly executing an import statement can seriously affect performance in some circumstances

What else?

List comprehensions are faster than building a list with a for loop

Use while 1 for an infinite loop (a single jump operation)

Exit a function as soon as you can to avoid executing code needlessly

Check out itertools (another module) to help build code that’s fast, memory-efficient, and elegant.

What else?

Use linked lists. I'm not kidding. They're fast.

The best way to sort items is to use keys and the default sort( ) method whenever possible:

```
import operator
my_list = [(”Nicole", ”Paige Brooks", "Entertainer"),
           ("Marco", "Polo", "General"),
           ("Ada", "Lovelace", "Scientist")]
my_list.sort(key=operator.itemgetter(0))
print(my_list)
```

Finally, try decorator caching!

Consider this simple function. Pass it a 36. So slow!

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n-2)
```

Finally, try decorator caching!

Try this version now.

```
import functools

@functools.lru_cache(maxsize=128)
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n-2)
```

Finally, try decorator caching!

In Python, a decorator function takes another function and extends its functionality

We denote these functions with the @ symbol

In the example we used the decorator functools.lru_cache function

We passed the maximum number of items to store in my cache at the same time as an argument

This took less than a second on my laptop!